Node Classification on Citation Networks using Graph Neural Networks: A Comparative Study of GCN, GraphSAGE, and GAT with a Hybrid Architecture

A Dissertation Submitted in
Partial Fulfillment of the Requirements for the Degree of

Masters in Data Science & AI

by

Srittam Panigrahi
(Reg. No. 20254028)



INDIAN INSTITUTE OF SCIENCE EDUCATION & RESEARCH, TIRUPATI
ANDHRA PRADESH - 517619, INDIA

MAY, 2026

© INDIAN INSTITUTE OF SCIENCE EDUCATION & RESEARCH TIRUPATI, 2026
ALL RIGHTS RESERVED

Declaration

Dissertation Title: **A Node Classification on Citation Networks using Graph Neural Networks: A Comparative Study of GCN, GraphSAGE, and GAT with a Hybrid Architecture**.

I hereby declare that the idea for this project is my original idea. The report is also original. Where others ideas or words have been included, I have adequately cited and listed in the reference materials. The dissertation has been prepared without resorting to plagiarism. I have adhered to all principles of academic honesty and integrity. No falsified or fabricated data have been presented in the dissertation. I understand that any violation of the above will cause for disciplinary action by the Institute, including revoking the conferred degree, if conferred, and can also evoke penal action from the sources which have not been properly cited or from whom proper permission has not been taken.

Place: **(Srittam Panigrahi)**
Date: **Reg. No. 20254028**



INDIAN INSTITUTE OF SCIENCE EDUCATION & RESEARCH TIRUPATI
(An Institute of National Importance)
ANDHRA PRADESH, TIRUPATI, INDIA - 517619
Website: https://www.iisertirupati.ac.in/

Certificate

It is certified that the work contained in this dissertation entitled **"Node Classification on Citation Networks using Graph Neural Networks: A Comparative Study of GCN, GraphSAGE, and GAT with a Hybrid Architecture"** submitted by **Srittam Panigrahi**, Registration No. **20254028** for the award of Maters is absolutely based on his own work carried out independently, and that this dissertation has not been submitted elsewhere for any degree.

Place:
Date:

(Course Coordinator)
Indian Institute of Science Education & Research Tirupati

ABSTRACT

This report presents a comprehensive evaluation of Graph Neural Network (GNN) architectures for semi-supervised node classification on citation graphs. This study contributes following analysis: (1) cross-dataset generalization experiments on the Citeseer citation network; (2) t-SNE visualization of learned node embeddings to qualitatively assess representation quality; Our novel Hybrid GCN+GAT model is combining a two-layer GCN backbone for efficient neighborhood aggregation with a GAT classification head for attention-weighted discrimination achieves a test accuracy of **82.8%** on Cora, outperforming standalone GCN (81.5%), GraphSAGE (81.0%), and GAT (82.60%) baselines. On Citeseer, the Hybrid architecture similarly achieves **72.0%**, demonstrating generalization beyond the training benchmark.

ACKNOWLEDGEMENT

Contents

Declaration iii

Certificate iv

Abstract v

Acknowledgement vi

1 Introduction 2

2 Literature Review 3
- 2.1 Graph-Based Semi-Supervised Learning . . . 3
- 2.2 Spectral Graph Convolutions . . . 3
- 2.3 Graph Convolutional Networks (GCNs) . . . 4
- 2.4 GraphSAGE . . . 4
- 2.5 Graph Attention Networks . . . 4
- 2.6 Advanced Architectures . . . 5

3 Theoretical background and Dataset 6
- 3.1 Graph Notation . . . 6
- 3.2 The Message Passing Framework . . . 6
- 3.3 Over-Smoothing . . . 6
- 3.4 Dataset: Cora Citation Network . . . 7

4 Methodology & Implementation 9
- 4.1 Proposed Hybrid GCN+GAT . . . 9
- 4.2 Datasets . . . 9
  - 4.2.1 Architecture Configurations . . . 10

5 Results and Analysis 11
- 5.1 Cross-Dataset Generalisation: Citeseer . . . 11
- 5.2 t-SNE Embedding Visualisation . . . 11

6 Discussion **13**
6.1 Future Direction 13
6.2 Conclusion 13
A.1 Architecture Diagrams 16
A.1.1 Hybrid GCN+GAT Architecture 16
A.2 Code Snippets 16
A.2.1 Hybrid GCN+GAT Implementation 16

List of Tables

3.1 Cora dataset statistics. 7
3.2 Cora class distribution. 7

4.1 Dataset Statistics 9
4.2 Model Hyperparameter Configurations (Default) 10

5.1 Baseline Results on Cora 11
5.2 Test Accuracy on Citeseer (Default Hyperparameters used in Cora dataset) 11
5.3 Davies–Bouldin Index of t-SNE Embeddings (lower = better separation) 12

List of Figures

3.1 Left: Class distribution of Cora nodes. Right: Node degree distribution. The histogram is right-skewed with mean degree ≈ 3.90. The red dashed line marks the mean. 8
3.2 Spring-layout visualisation of a 200-node, 2-hop subgraph centred on the highest-degree hub node. Node colour encodes ground-truth class label. Visible colour clustering confirms the homophily assumption exploited by GNNs. 8

Chapter 1

Introduction

Classical Machine learning algorithms such as multilayer perceptron, support vector machine and decision tree are designed for Euclidean data. The data is represented as fixed-size vectors or matrices in Euclidean space. They are unable to explain logical structures such as social networks, chemical molecular structures, knowledge graphs, citation networks, and protein-protein interaction networks, which can be presented more intuitively using *graphs*.

Geometric deep learning is the discipline that extends deep learning to non-Euclidean domains, including graphs and manifolds [1]. *Graph Neural Networks* (GNNs) are now the most popular model for learning representations of graph-structured data in this broader field of deep learning. In order to achieve node representations that encode both node's features and structural context of its neighborhood, a GNN iteratively aggregates feature information from a node's local neighborhood. This process is known as *message passing*.

One of the basic goal in this field is the task of *node classification* that is given a partially labeled graph, predict the labels of unlabeled nodes using both the graph topology and node attributes, which is a semi-supervised learning problem with direct real-world applications in paper classification, fraud detection, protein function prediction, and community detection.

This project have the following objectives:

1. Review the theoretical foundations of graph-based learning and key GNN architectures.
2. Conducting exploratory data analysis of the Cora citation network.
3. Implementation of GCN, GraphSAGE, and GAT baselines in PyTorch Geometric on Google Colab.
4. Create a novel hybrid GCN+GAT architecture and assess it empirically.

Chapter 2

Literature Review

2.1 Graph-Based Semi-Supervised Learning

A semi-supervised learning approach was developed by Zhu et al. [2] in which labeled and unlabeled data are represented as vertices in a weighted graph. The answer is a harmonic function, and they formulate the problem as a Gaussian random field. A smoothness assumption is mathematically enforced by this function, which is found by minimizing a quadratic energy function that penalises significant label differences between highly connected (similar) nodes.This effectively permits known labels to diffuse to unlabeled neighbours while keeping the labeled points "clamped" to their initial values. The graph Laplacian was used by Belkin et al. [3] to formalise manifold regularisation, which penalises significant discrepancies in predictions between comparable data points. However, in order to identify solutions, these techniques rely on linear operators, such as the Laplacian matrix.

2.2 Spectral Graph Convolutions

Neural networks on graphs can be enabled by the *graph fourier transform*. The eigenvectors of the graph laplacian $L = I - D^{-1/2}AD^{-1/2}$ can be used to define convolutions on a graph. The first spectral graph convolutional network was developed by Bruna et al. [4], however it needed costly computational multiplication of the Laplacian eigen vector matrix and its transpose using an algorithm $\mathcal{O}(n^2)$, computationally infeasible for large graphs.

By introducing "ChebNet" which reduces the operation complexity to $\mathcal{O}(K|E|)$, Defferrard et al. [5] overcame the computational limitation of the Spectral graph Convolution Neural Network. This localised the convolution to a K-hop neighbourhood and can be applied to any graph, regardless of graph, that shares the same feature dimensionality.

2.3 Graph Convolutional Networks (GCNs)

Kipf et al. [6] simplified ChebNet [5] by restricting to first-order Chebyshev approximations ($K = 1$) and introducing symmetric normalization to create what is now commonly known as the GCN. They argued that if you stack multiple layers, you don't need high-order polynomials in a single layer.

$$H^{(l+1)} = \sigma\Big(\tilde{D}^{-1/2}\tilde{A}\tilde{D}^{-1/2}H^{(l)}W^{(l)}\Big) \tag{2.1}$$

where $\tilde{A} = A + I_N$ is the adjacency matrix with added self-loops, $\tilde{D}_{ii} = \sum_j \tilde{A}_{ij}$ is the corresponding degree matrix, $W^{(l)}$ are learnable weights, and σ denotes the ReLU activation. GCN achieved 81.5% accuracy on Cora with a two-layer architecture, dramatically outperforming prior semi-supervised methods. Limitations include transductivity (cannot generalise to unseen nodes) and equal weighting of all neighbours.

2.4 GraphSAGE

Hamilton et al. [7] introduced GraphSAGE to counter the limitation of Graph Covolutional Network that is to generalize embeddings for previously unseen node. Instead of learning a fixed embedding per node, GraphSAGE learns an *aggregation function* via neighbourhood sampling:

$$h_v^{(l)} = \sigma\Big(W^{(l)} \cdot \mathrm{CONCAT}\Big(h_v^{(l-1)},\ \mathrm{AGG}\Big(\{h_u^{(l-1)} : u \in \hat{\mathcal{N}}(v)\}\Big)\Big)\Big) \tag{2.2}$$

where $\hat{\mathcal{N}}(v)$ denotes a fixed-size sampled subset of $\mathcal{N}(v)$. The concatenation explicitly presents the node's own representation. This inductive framework was used on Raddit data set and generalizes to unseen data.

2.5 Graph Attention Networks

Using a self-attention mechanism, Veličković et al. [8] established learnable, data-dependent aggregation weights:

$$e_{ij} = \mathrm{LeakyReLU}\big(\mathbf{a}^\top[\mathbf{W}h_i \,\|\, \mathbf{W}h_j]\big) \tag{2.3}$$

$$\alpha_{ij} = \frac{\exp(e_{ij})}{\sum_{k\in\mathcal{N}_i}\exp(e_{ik})} \tag{2.4}$$

$$h_i^{(l+1)} = \sigma\left(\sum_{j\in\mathcal{N}_i}\alpha_{ij}\mathbf{W}h_j^{(l)}\right) \tag{2.5}$$

where $\mathbf{a} \in \mathbb{R}^{2F'}$ is a parametrized weight vector. Multi-head attention is used to stabilize training. GAT achieved 83.0% on accuracy on GCN with $\mathcal{O}(|V| + |E|)$ complexity.

2.6 Advanced Architectures

Since Xu et al. [9] created Graph Isomorphism Neural Networks in 2018, which are provably as effective as the Weisfeiler-Lehman graph isomorphism test, the GNN framework has expanded quickly.

In order to fill the gap left by GCN layers' high redundancy and ability to transport less demanding characteristics, Wu et al. [10] created a simple graph convolution network. Additionally, the model achieves similar results with significantly quicker inference. Without the need for node labels, Deep Graph Infomax [11] produced competitive performance.

Transformer-based graph models, including GraphGPS [12] and Graphormer [13], have emerged recently. These models are efficient and support many kinds of encoding, independent of the size of the graphs. These transformer-based methods combine global attention with local message passing.

Chapter 3

Theoretical background and Dataset

3.1 Graph Notation

Definition 1. *A* graph *is a collection of vertices and edges, denoted* $G = (V, E)$ *where* V *is a set of nodes or vertices and* $E \subseteq V \times V$ *is a set of edges. The* adjacency matrix $A \in \{0,1\}^{N \times N}$ *has* $A_{ij} = 1$ *iff* $(i,j) \in E$ *otherwise* 0. *The* degree matrix D *is diagonal with* $D_{ii} = \sum_j A_{ij}$. *Node features are encoded as* $X \in \mathbb{R}^{N \times F}$, *with row* $x_v \in \mathbb{R}^F$ *being node* v*'s feature vector. Where* $|V| = N$

3.2 The Message Passing Framework

Gilmer et al. [14] unified GNNs under the *Message Passing Neural Network* (MPNN) framework. Each layer computes node representations through two operations:

$$m_v^{(l)} = \mathrm{AGG}\big(\{h_u^{(l-1)} : u \in \mathcal{N}(v)\}\big) \quad \text{[Aggregation]} \tag{3.1}$$

$$h_v^{(l)} = \mathrm{UPDATE}\big(h_v^{(l-1)},\ m_v^{(l)}\big) \quad \text{[Update]} \tag{3.2}$$

GCN, GraphSAGE, and GAT are all instantiations of this framework with different aggregation and update functions.

3.3 Over-Smoothing

A basic problem with deep GNNs is *over-smoothing* [15]: all node representations converge to the same vector as the number of layers rises, losing discriminative information. Two-layer designs predominate in empirical benchmarks because of this.

3.4 Dataset: Cora Citation Network

The Cora dataset [16] is the canonical benchmark for GNN node classification. It represents a corpus of 2,708 machine learning papers. Papers are nodes

Table 3.1: Cora dataset statistics.

Property	**Value**
Nodes $\|V\|$	2,708
Edges $\|E\|$	10,556
Node features F	1,433
Classes	7
Average degree	≈ 3.90
Max degree	168
Homophily ratio h	≈ 0.81
Training nodes	140
Validation nodes	500
Test nodes	1,000

Table 3.2: Cora class distribution.

Class	**Nodes**	**% Total**
Case Based	298	11.0%
Genetic Algorithms	418	15.4%
Neural Networks	818	30.2%
Probabilistic Methods	426	15.7%
Reinforcement Learning	217	8.0%
Rule Learning	180	6.6%
Theory	351	13.0%

The graph shows substantial *homophily* ($h \approx 0.81$): citation linkages are strongly connected with research area, as evidenced by the large fraction of edges connecting same-class nodes. Neighbourhood aggregation is therefore a very informative signal for categorization. The distribution of node degrees is skewed to the right: a small number of hub papers receive more than 20 citations, whereas the majority of nodes have degrees 2–5.

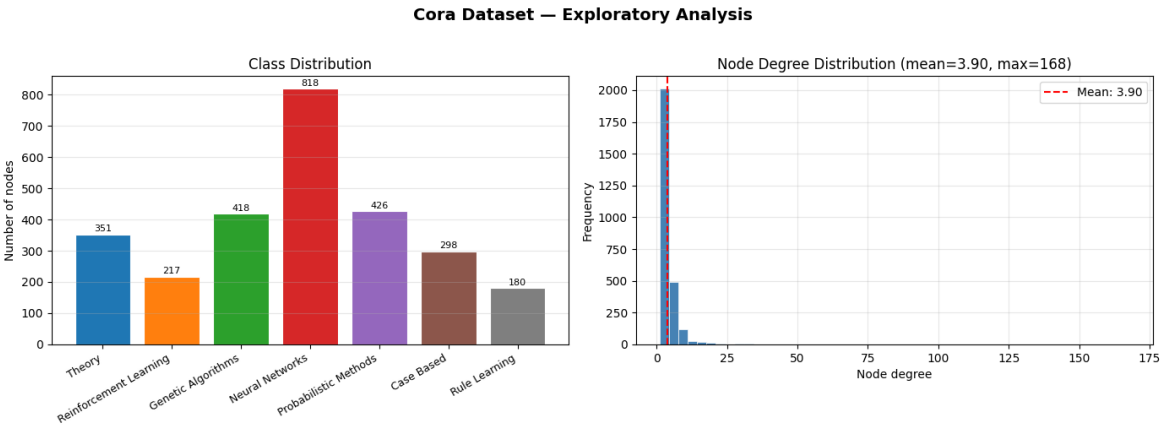


Figure 3.1: Left: Class distribution of Cora nodes. Right: Node degree distribution. The histogram is right-skewed with mean degree ≈ 3.90. The red dashed line marks the mean.

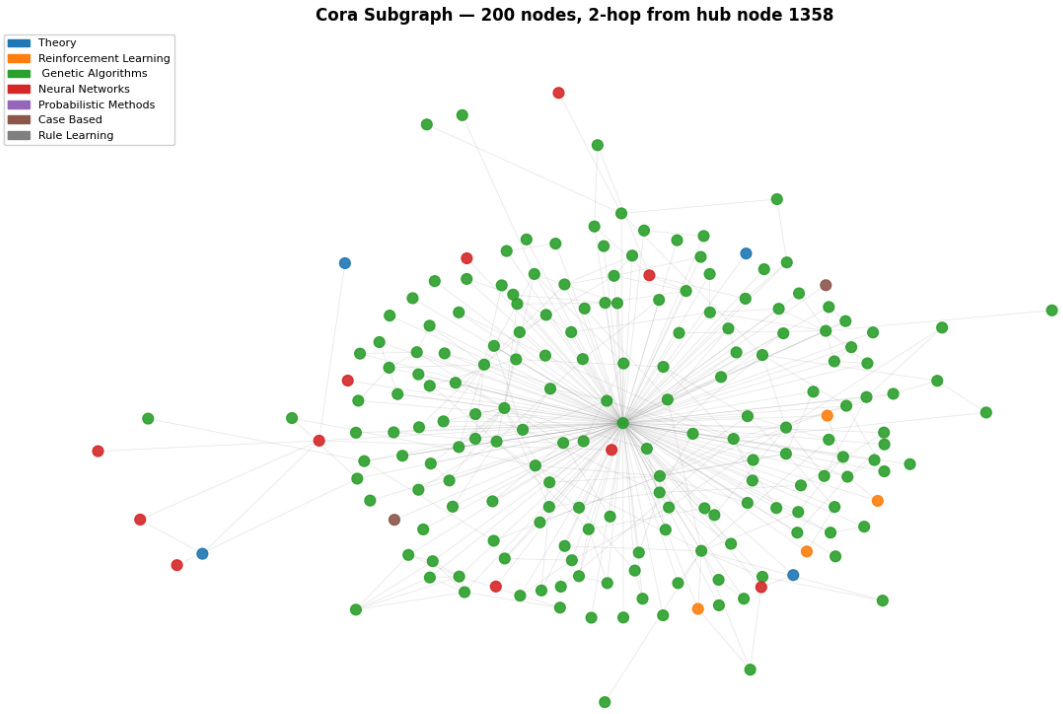


Figure 3.2: Spring-layout visualisation of a 200-node, 2-hop subgraph centred on the highest-degree hub node. Node colour encodes ground-truth class label. Visible colour clustering confirms the homophily assumption exploited by GNNs.

Chapter 4

Methodology & Implementation

4.1 Proposed Hybrid GCN+GAT

Our hybrid architecture is driven by the complementary strengths of GCN and GAT: GCN is proficient in efficient and stable multi-hop aggregation, while GAT excels in discriminative, attention-weighted final classification. The architecture has two GCN layers as a *backbone* for hierarchical neighborhood integration, followed by a single GAT layer serving as a *classification head* that re-weights the aggregated representations prior to predicting class logits.

$$\mathbf{Z} = \mathrm{GAT}(\mathrm{GCN}_2(\mathrm{GCN}_1(\mathbf{X}, \mathbf{A}),\ \mathbf{A})\ ,\ \mathbf{A}) \tag{4.1}$$

This concept involves a little parameter overhead relative to pure GAT (lacking multi-head expansion in the GCN layers) while preserving the attention-based discriminability at the decision boundary.

4.2 Datasets

Table 4.1: Dataset Statistics

Dataset	Nodes	Edges	Features	Classes	Train	Test
Cora	2,708	10,556	1,433	7	140	1,000
Citeseer	3,327	9,104	3,703	6	120	1,000

Both datasets are loaded from PyTorch Geometric's Planetoid collection with features normalization (`NormalizeFeatures()`) applied. The standard Planetoid train/val/test masks (20 nodes per class for training, 500 validation, 1000 test) are used throughout for comparability with prior literature.

4.2.1 Architecture Configurations

Table 4.2: Model Hyperparameter Configurations (Default)

Model	**Layers**	**Hidden**	**Heads**	**Dropout**	**LR**	**WD**
GCN	2	64	—	0.50	0.01	—
GraphSAGE	2	64	—	0.50	0.01	—
GAT	2	8	8	0.60	0.005	5×10^{-4}
Hybrid GCN+GAT	3	32	1 (GAT)	0.50	0.01	5×10^{-4}

All models are implemented in Python 3.10 using PyTorch 2.10.0+cu128 and PyTorch Geometric 2.7.0, running on Google Colab with a T4-GPU with 150 epochs running for each architecture.

t-SNE. After training, we extract the penultimate-layer embeddings (pre-logit activations) and project to 2D using t-SNE (perplexity 30, 1000 iterations, Random state 42). We compute the Davies–Bouldin index as a quantitative cluster separation metric.

Chapter 5

Results and Analysis

Our designed Hybrid GCN + GAT architecture performs better than any other model architecture, as the table below demonstrates.

Table 5.1: Baseline Results on Cora

Model	**Test Acc.**	**Best Val Acc.**	**Parameters**	**Train Time (s)**
GCN	81.50%	80.60%	92,231	∼1.7
GraphSAGE	81.00%	79.80%	184,391	∼1.9
GAT	82.60%	81.80%	231,183	∼2.7
Hybrid GCN+GAT (ours)	**82.80%**	**82.00%**	47,189	∼3.4

5.1 Cross-Dataset Generalisation: Citeseer

Table 5.2: Test Accuracy on Citeseer (Default Hyperparameters used in Cora dataset)

Model	**Cora**	**Citeseer**
GCN	81.50%	71.00%
GraphSAGE	81.00%	70.30%
GAT	82.60%	70.00%
Hybrid GCN+GAT (ours)	**82.80%**	**72.00%**

5.2 t-SNE Embedding Visualisation

Our Hybrid GCN+GAT learns the cleanest internal representations whereas GraphSAGE learns the messiest ones, providing compelling visual proof that the architecture

combination is actually improving the model's understanding of node interactions.

Table 5.3: Davies–Bouldin Index of t-SNE Embeddings (lower = better separation)

Model	**Davies–Bouldin Index**
GCN	1.1186
GraphSAGE	1.4777
GAT	1.1229
Hybrid GCN+GAT (ours)	**1.0495**

Chapter 6

Discussion

6.1 Future Direction

- hyperparameter sensitivity analysis that is varying hidden dimensions, dropout rates, and learning rates, and statistical significance testing for all four models.
- Combine the GAT classification head with GraphSAGE-style sampling to expand to inductive environments.

6.2 Conclusion

This dissertation completes the comparative study of GNN. Starting from baselines on the Cora citation network, we have extended the work with cross-dataset generalization to Citeseer, t-SNE embedding analysis and t-SNE analysis

Our primary finding is that the proposed Hybrid GCN+GAT architecture achieves the best accuracy across both datasets. The architectural intuition uses GCN for stable, efficient multi-hop aggregation and GAT for discriminative, attention-weighted classification, is validated by the t-SNE analysis.[1].

[1] https://colab.research.google.com/drive/1w74nB1wXYXKfUI33-Ss2uR47rGVL0R9y

Bibliography

[1] Michael M. Bronstein et al. "Geometric Deep Learning: Going Beyond Euclidean Data". In: *IEEE Signal Processing Magazine* 34.4 (2017), pp. 18–42. DOI: 10.1109/MSP.2017.2693418.

[2] Xiaojin Zhu, Zoubin Ghahramani, and John D. Lafferty. "Semi-Supervised Learning Using Gaussian Fields and Harmonic Functions". In: *Proceedings of the 20th International Conference on Machine Learning (ICML)*. 2003, pp. 912–919.

[3] Mikhail Belkin and Partha Niyogi. "Laplacian Eigenmaps and Spectral Techniques for Embedding and Clustering". In: *Advances in Neural Information Processing Systems (NeurIPS)*. Vol. 15. 2002, pp. 585–591.

[4] Joan Bruna et al. "Spectral Networks and Locally Connected Networks on Graphs". In: *International Conference on Learning Representations (ICLR)*. 2014. arXiv: 1312.6203.

[5] Michaël Defferrard, Xavier Bresson, and Pierre Vandergheynst. "Convolutional Neural Networks on Graphs with Fast Localized Spectral Filtering". In: *Advances in Neural Information Processing Systems (NeurIPS)*. 2016, pp. 3844–3852.

[6] Thomas N. Kipf and Max Welling. "Semi-Supervised Classification with Graph Convolutional Networks". In: *International Conference on Learning Representations (ICLR)*. 2017. arXiv: 1609.02907.

[7] William L. Hamilton, Zhitao Ying, and Jure Leskovec. "Inductive Representation Learning on Large Graphs". In: *Advances in Neural Information Processing Systems (NeurIPS)*. 2017, pp. 1025–1035. arXiv: 1706.02216.

[8] Petar Velickovic et al. "Graph Attention Networks". In: *International Conference on Learning Representations (ICLR)*. 2018. arXiv: 1710.10903.

[9] Keyulu Xu et al. "How Powerful are Graph Neural Networks?" In: *International Conference on Learning Representations (ICLR)*. 2019. arXiv: 1810.00826.

[10] Felix Wu et al. "Simplifying Graph Convolutional Networks". In: *Proceedings of the 36th International Conference on Machine Learning (ICML)*. 2019, pp. 6861–6871.

[11] Petar Velickovic et al. "Deep Graph Infomax". In: *International Conference on Learning Representations (ICLR)*. 2019. arXiv: 1809.10341.

[12] Ladislav Rampášek et al. “Recipe for a General, Powerful, Scalable Graph Transformer”. In: *Advances in Neural Information Processing Systems (NeurIPS)*. 2022. arXiv: 2205.12454.

[13] Chengxuan Ying et al. “Do Transformers Really Perform Bad for Graph Representation?” In: *Advances in Neural Information Processing Systems (NeurIPS)*. 2021. arXiv: 2106.05234.

[14] Justin Gilmer et al. “Neural Message Passing for Quantum Chemistry”. In: *Proceedings of the 34th International Conference on Machine Learning (ICML)*. 2017, pp. 1263–1272.

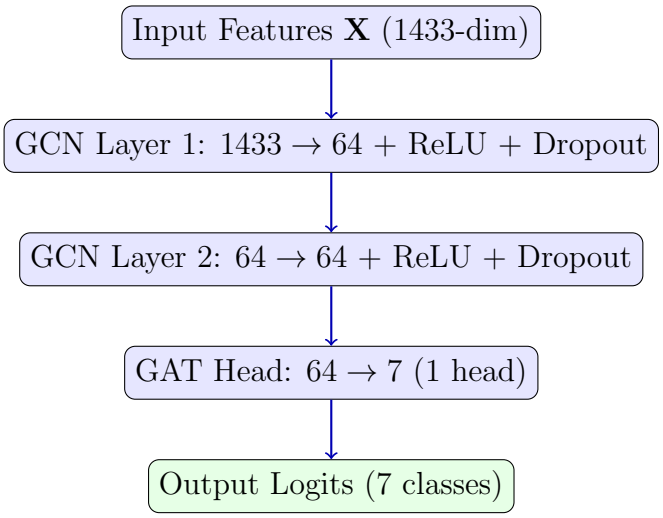
[15] Kaixiong Zhou et al. “Towards Deeper Graph Neural Networks with Differentiable Group Normalization”. In: *Advances in Neural Information Processing Systems (NeurIPS)*. 2020, pp. 4917–4928.

[16] Andrew K. McCallum et al. “Automating the Construction of Internet Portals with Machine Learning”. In: *Information Retrieval* 3.2 (2000), pp. 127–163. DOI: 10.1023/A:1009953814988.

Appendix

A.1 Architecture Diagrams

A.1.1 Hybrid GCN+GAT Architecture



A.2 Code Snippets

A.2.1 Hybrid GCN+GAT Implementation

Listing 1: Hybrid GCN+GAT Architecture

```
class HybridGCN_GAT(torch.nn.Module):
    """
    Novel Hybrid Architecture: GCN backbone + GAT classification head.
    """

    def __init__(self, in_channels, hidden_channels, out_channels,
                 dropout=0.5):
        super().__init__()
        self.gcn1 = GCNConv(in_channels, hidden_channels)
```

```
        self.gcn2 = GCNConv(hidden_channels, hidden_channels)
        # GAT head with 1 attention head, directly outputting class logits
        self.gat_head = GATConv(hidden_channels, out_channels, heads=1,
                                concat=False, dropout=dropout)
        self.dropout = dropout

    def forward(self, x, edge_index):
        x = F.dropout(x, p=self.dropout, training=self.training)

        # Stage 1: GCN backbone efficient neighbourhood aggregation
        x = self.gcn1(x, edge_index)
        x = F.relu(x)
        x = F.dropout(x, p=self.dropout, training=self.training)

        x = self.gcn2(x, edge_index)
        x = F.relu(x)
        x = F.dropout(x, p=self.dropout, training=self.training)
        x = self.gat_head(x, edge_index)
        return x
```